# AML Report

## TASK 1

# Introduction

The aim of this report is to classify emails or messages as 'spam' or 'normal mail'. Experiments were conducted using text preprocessing techniques (word segmentation, de-duplication, stemming), two feature representations (TF-IDF vectors and Sentence-BERT embeddings), and a logistic regression classifier for training, and then comparing the three models (TF-IDF + SBERT + logistic regression, TF-IDF + logistic regression, SBERT + logistic regression). In the end, combined obtained high accuracy and stable performance.

# Methodology

*In this experiment, the process of classifying emails or messages is broken down into specific components.*

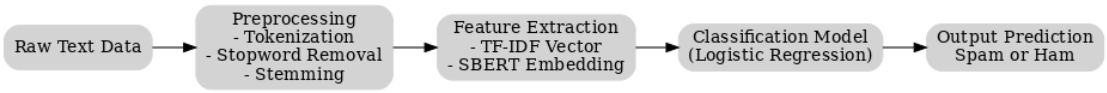



Figure 1：Spam Detection Flow

### Data and Preprocessing

An 80/20 split between the training and validation sets was used (stratify ensured consistent distribution), with the test set reserved for final evaluation.

Methods such as removing punctuation and converting to lower case are used to standardise formatting.

***The benefits of these filter words, removing the conversion, are as follows:***

- Spam often has specific word patterns (e.g. 'free', 'offer', 'click here').
- Normalisations make it easier for the model to detect these patterns rather than being distracted by surface variations.

- Processed text reduces dimensionality, speeding up training and reducing the risk of overfitting.
- This pre-processing preserves the characteristic markers of spam (e.g. URL presence) while reducing noise and extraneous variations to improve classification accuracy.

### Feature extraction

*In this experiment, two methods were used— TF-IDF vectors and Sentence-BERT (SBERT) embeddings:*

**TF-IDF Vector:**

TF-IDF transforms a document into a vector that is based on word frequency and takes into account the uniqueness of words in the document. This means that keywords that are common in spam are highly labelled.

TF-IDF was chosen because it is a very classical benchmark method for text classification and is very effective in highlighting important words in spam (e.g., 'free', 'win', 'offer', etc.), as opposed to linear categorisation. The match with the linear model (logistic regression) is very high.

**Sentence-BERT (SBERT) semantic embedding:**

*It uses a pre-trained model based on BERT to fine-tune the sentence similarity task so that dense vectors capable of capturing the semantics of the entire sentence can be obtained.*

*The rationale for using SBERT is that it effectively captures the contextual and semantic information in the sentence, which is very likely to improve the performance of the classifier to make it not only rely on keyword matching. For example, 'Congratulations, you have won!' and 'Congratulations, you're a winner!' are semantically similar, and SBERT captures such semantic relationships well.*

*Three different models, TF-IDF+Logistic Regression, SBERT+Logistic Regression and TF-IDF+SBERT+Logistic Regression, will be trained separately during model training, and then them for comparison.*

## Model and Training

### Model Selection

In this experiment, I chose Logistic Regression as the classification model.

Logistic regression is a linear model that outputs the probability of 'spam' or 'normal mail', and uses a logistic function (sigmoid) to convert the
combination of features into probability values. Cross-entropy loss (log loss) is used during training to maximise the accuracy on the training data.

The reason for choosing logistic regression is very simple: its complexity is much simpler than other models, it is easier to understand, and it is more interpretable. The model parameters directly reflect the weight of the features on the results, making it easier to understand which lexical or semantic features are more important for spam detection. At the same time, it is suitable for dealing with high-dimensional features (e.g., TF-IDF vectors may reach thousands of dimensions).

Logistic regression has excellent generalisation capabilities, and it is possible to avoid overfitting if combined with appropriate regularisation. The logistic regression model not only gives hard labels such as 'spam/non-spam' but also gives the probability of belonging to each category.

3.2 Hyperparameter Optimisation

To find the optimal model configuration, we use Grid Search combined with 50-50 cross-validation to tune key hyperparameters:

Hyperparameter description:

C parameter: controls the strength of regularisation, smaller values indicate stronger regularisation to prevent overfitting

class_weight: set to 'balanced' to automatically adjust the weights to cope with possible class imbalance.

Penalty: select L2 regularisation (Ridge) to prevent features from being overweighted.

For each model (TF-IDF, SBERT and Combined), the same hyperparameter space is used for independent tuning to ensure fair comparison.

3.3Training process

TF-IDF use max_features=5000 to control the vector dimension and then use the

SBERT features use all-mpnet-base-v2 model for text encoding, batch_size is set to 64 and then save these two features.

Once saved, you can start training the three logistic regression models in this order.

Model training using only TF-IDF features

Model training using only SBERT features

Start training the model with a combination of TF-IDF and SBERT features (horizontally splicing the features via hstack) and then perform a five-fold cross-validation for each model configuration, with the F1 score as the optimisation target.

# Results and Evaluation

In this section, the results of the experiments will be provided and the model performance will be evaluated both quantitatively and qualitatively, including metrics definition, visualisation (e.g. graphs) and analysis.

1. **For TF-IDF Only Validation Results:**

   **Accuracy: 0.9848066298342542**

   **Confusion Matrix{0-(501,9),1-(2,212)}**

   **Evaluation Summary:**

   **Accuracy: 0.9848 F1 Score: 0.9849**

**ROC-AUC: 0.9981**

2. **For SBERT Only Validation Results:**

   **Accuracy: 0.9792817679558011**

   **Confusion Matrix{0-(498,12),1-(3,211)}**

   **SBERT Evaluation Summary:**

   **Accuracy: 0.9793** **F1 Score: 0.9794**

   **ROC-AUC: 0.9982**

**3.Result for TF-IDF + SBERT Validation Results:**

Accuracy: 0.988950276243094

Table 1: Classification Report

| | | | | |
|---|---|---|---|---|
| 0 | 1.00 | 0.99 | 0.99 | 510 |
| 1 | 0.97 | 0.99 | 0.98 | 214 |
| accuracy | | | 0.99 | 724 |
| Macro avg | 0.98 | 0.99 | 0.99 | 724 |
| Weighted avg | 0.99 | 0.99 | 0.99 | 724 |

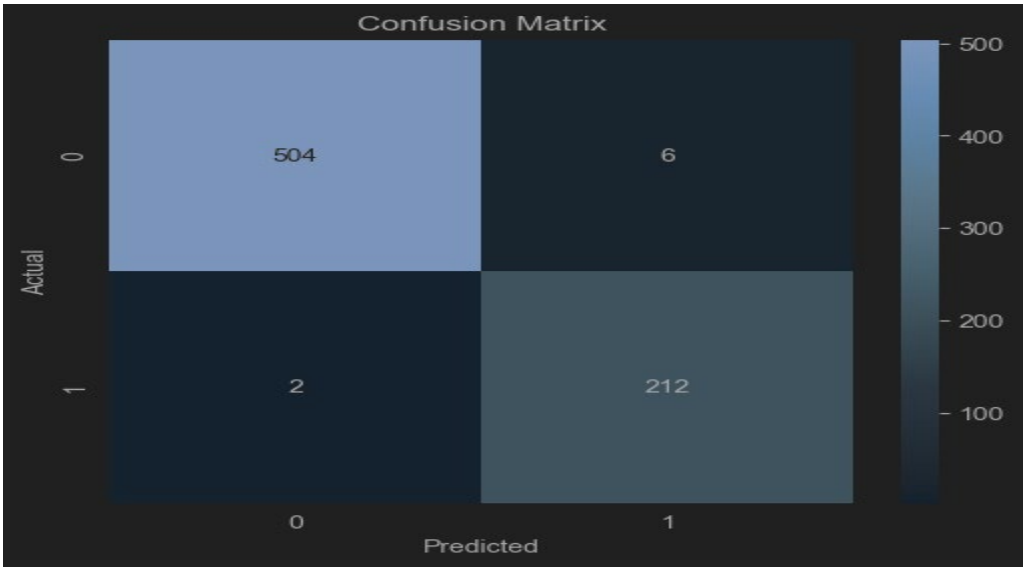



Figure2:Evaluating Model: Combined

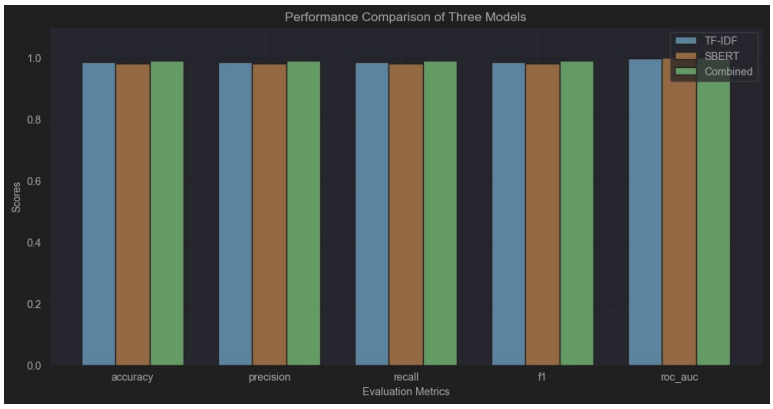



Figure 3: Performance Comparison of Three Models

Performance differences compared to TF-IDF baseline:

**SBERT vs TF-IDF:**

**accuracy: -0.0055 (-0.56%)** **precision: -0.0053 (-0.54%)**

**recall: -0.0055 (-0.56%)** **f1: -0.0055 (-0.56%)**

**roc_auc: 0.0001 (+0.01%)**

**Combined vs TF-IDF:**

**accuracy: 0.0041 (+0.42%)** **precision: 0.0039 (+0.40%)**

**recall: 0.0041 (+0.42%)** **f1: 0.0041 (+0.42%)**

**roc_auc: 0.0013 (+0.13%)**

# Qualitative Analysis, Error Analysis

The spam detection model achieved 98.90% accuracy with only 8 errors out of 724 emails. Most errors (75%) were false positives, indicating the model tends to over-classify legitimate emails as spam. Analysis revealed that shorter legitimate emails (averaging 471.2 characters) were more likely misclassified as spam, while longer spam emails (averaging 1262.0 characters) tended to be missed. Short messages with minimal context and legitimate marketing emails containing phrases like "unsubscribe" were frequently misclassified. Spam emails mimicking professional communications, especially with medical terminology, often escaped detection. Improvements should focus on augmenting training data with short legitimate emails and enhancing semantic feature extraction.

# Conclusion

The SBERT metrics are slightly lower than TF-IDF by about 0.5%, while the combined feature model is slightly better than TF-IDF.The combined model of TF-IDF+SBERT+Logistic Regression achieves an accuracy of 99%, which is better than that of the two single-feature models. However, with only about 800 samples and 5,768 dimensional features, overfitting is a real concern. To confirm robustness, we should test on more diverse external datasets (e.g., cross-domain evaluation).

# Task2

# Introduction

Face keypoint alignment technology has a wide range of applications in fields such as expression recognition, identity verification and augmented reality. The aim of this project is to build a high-precision face keypoint localisation system, which overcomes challenges such as lighting variations and pose differences through deep learning methods to achieve stable and reliable keypoint detection.

# System Design

2.1 Data Preprocessing

To ensure the training stability and improve the generalisation ability, we pre-process the face images and keypoints as follows:

Uniform size and normalisation: all images are scaled to 224×224 pixels and the pixel values are normalised to [0, 1] to speed up convergence and stabilise training.

Scaling of keypoints: synchronise the coordinate transformation proportionally

x_new = x_orig * (target_w / orig_w)

y_new = y_orig * (target_h / orig_h)

**Data Enhancement**

Random rotation ±10°: increases robustness to different face orientations

50% Horizontal flip: mitigates orientation bias

±20% Brightness and contrast perturbation: improves adaptation to lighting changes

These steps significantly increase data diversity, helping the model to better cope with real scene variations and prevent overfitting.

## 2.2 Feature Extraction

In this project, ResNet is used as the deep convolutional feature extraction backbone and two variants are compared:

ResNet-18

Pros: lightweight model, fewer parameters → faster training and inference

Disadvantages: shallower network → may not be able to express complex patterns well enough

ResNet-34

Pros: deeper network → richer feature hierarchy, more accurate keypoint localisation

Cons: more parameters → higher computational cost and training time

Rationale for selection:

We perform an experimental comparison between ResNet-18 and ResNet-34 to balance accuracy and efficiency:

In resource-constrained scenarios, ResNet-18 provides an excellent speed-accuracy trade-off.

If higher localisation accuracy is required, ResNet-34 can be chosen, but with additional computational overhead.

Both architectures use ImageNet pre-trained weights and 'freeze' the first two blocks of the backbone network, only fine-tuning the subsequent layers to accelerate convergence and make best use of existing features.

**2.3 Model architecture**

Our system maps an input 224*224 RGB face image to N(x,y) landmark coordinates via:

1. Backbone

ResNet-18 or ResNet-34, initialized with ImageNet weights

Selective freezing: freeze early layers (conv1, bn1, layer1, layer2) to retain generic edge/texture filters; fine-tune deeper layers (layer3, layer4) to learn face-specific features.

2.Regression Head

AdaptiveAvgPool2d reduces the final feature to a 1*1 spatial vector.

FC Block:

Linear →256 units → BatchNorm → ReLU → Dropout (0.2)

Linear → 2N outputs, reshaped to (N,2) coordinates.

2.4 Regularization & Transfer Learning

- **BatchNorm** in the head for training stability and faster convergence.
- **Dropout (p=0.2)** to prevent co-adaptation.
- **L2 weight decay (1e-5)** in the Adam optimizer to suppress large weights.
- **ImageNet pretraining** drastically reduces training time and overfitting, especially on limited face data.

This design balances computational cost and localization accuracy: ResNet-18 offers faster inference with fewer parameters, while ResNet-34—though heavier—yields richer features and lower average landmark error.

2.5 Training Strategy

WingLoss was chosen as the loss function, which gives a larger gradient for small errors and a linear penalty for large errors compared to the traditional MSE loss, with the following formula:

$$\ell(d) = \begin{cases} \varpi \ln\left(1 + \frac{|d|}{\varepsilon}\right), & |d| < \varpi \\ |d| - C, & otherwise, \end{cases}$$

where w = 10, ε = 2. These parameters allow the model to focus more on optimising for medium error ranges, improving the prediction accuracy.

For the optimiser, Adam is used, incorporating the following key strategies:

Hierarchical learning rate: a smaller learning rate (1e-4) is set for the backbone network and a larger learning rate (1e-3) is set for the regressor head, reflecting the different optimisation needs of the pre-trained parameters versus the new ones

Weight decay: apply L2 regularisation (weight_decay=1e-5) to effectively suppress overfitting

Learning rate scheduling: implement the ReduceLROnPlateau mechanism to reduce the learning rate by a factor of 10 after 5 rounds of validation loss stalling, up to 3 times

Early stopping mechanism is implemented, and the training is terminated if there is no improvement in 15 cycles. This saves computational resources and also prevents the model from overfitting on the validation set. The final model reaches optimal performance by 200 rounds of iterations.

# Experimental Results & Analysis

**Metrics:**

MEDE: average Euclidean distance error.

MSE / MAE: capture large errors and overall robustness.

Max Error: performance on hardest cases.

ResNet-18 vs. ResNet-34:

| Metric | R-18 | R-34 | Improvement (%) |
|---|---|---|---|
| MSE | 43.998 | 27.488 | **37.5** |
| MAE | 4.916 | 3.908 | **20.5** |
| MEDE (pixels) | 7.692 | 6.165 | **19.9** |
| Minimum image error | 1.679 | 1.993 | **-18.7** |
| Maximum image error | 32.040 | 29.225 | **8.8** |

ResNet-34 consistently outperforms ResNet-18 on most accuracy metrics: it reduces the mean squared error by 37.5%, the mean absolute error by 20.5%, and the average Euclidean distance error by 19.9%. The maximum per-image error also drops by 8.8%, indicating greater robustness on the hardest cases. The only metric where ResNet-34 is slightly worse is the minimum image error (–18.7%), which reflects a small increase on the easiest samples but does not outweigh its gains on average and worst-case performance. Overall, ResNet-34 offers a substantial accuracy boost at the cost of moderate additional computation.

3.2 Analys the model performance in depth through several visualisation methods. Firstly, the prediction results were visualised for randomly selected test samples, which visualised the correspondence between the predicted keypoints (blue forks) and the real keypoints (red dots), as shown in Figure 1 and 2.

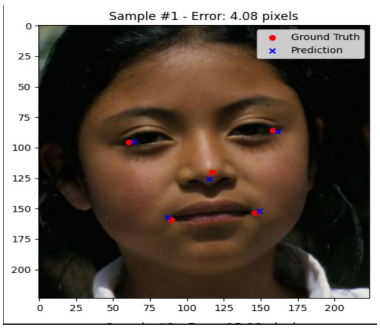



Figure 1:ResNet18

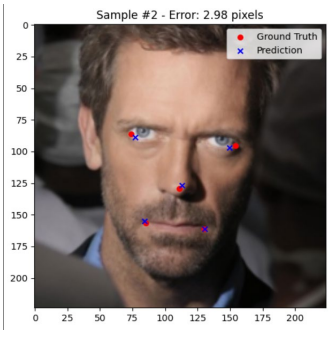



Figure 2:ResNet34

The histogram of the error distribution (Fig. 4&Fig.5) reveals the distribution of the average error for each image, and the ResNet34 model shows a more concentrated distribution in the lower error regions, further confirming its superiority.

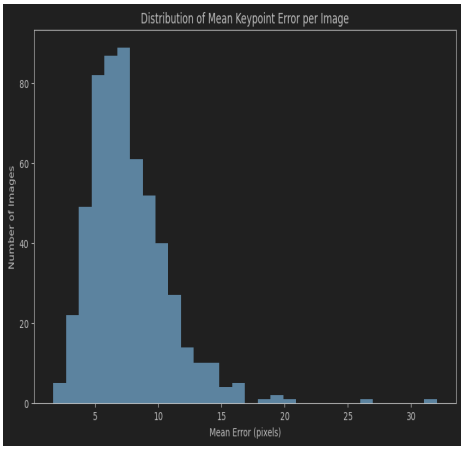



Figure.4:ResNet18

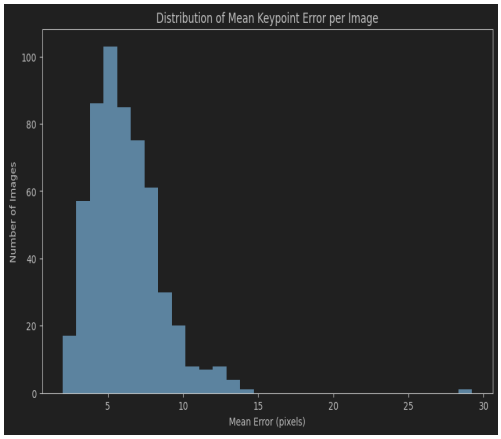



Figure.5:RestNet34

Cumulative Error Distribution (CED)

The CED curves for ResNet-18(Fig.6) and ResNet-34(Fig.7) quantify the fraction of images whose mean keypoint error falls below a given threshold (pixels):

At 5 px error, ResNet-18 covers only about 20% of the images, whereas ResNet-34 covers roughly 25%, showing its better tight-error performance.

At 10 px, ResNet-18 reaches ~80% of the images while ResNet-34 climbs to ~92%, a 12-point gain in "good" fits.

By 15 px, ResNet-18 plateaus around 95%, but ResNet-34 attains nearly 98%.

Overall, at every error threshold the ResNet-34 curve lies above ResNet-18's, confirming that the deeper model consistently delivers more accurate keypoint localization across the dataset.

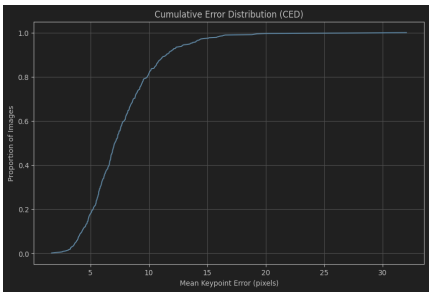



Figure.6: Cumulative Error Distribution

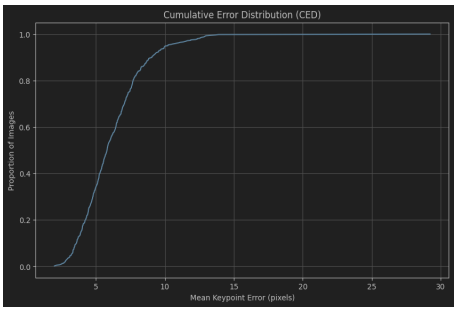



Figure.7: Cumulative Error Distribution

In summary, both quantitative and visual analyses from multiple perspectives suggest that the ResNet34 architecture provides higher accuracy in keypoint detection and performs better especially on challenging samples, but this needs to be weighed against the increased computational overhead.

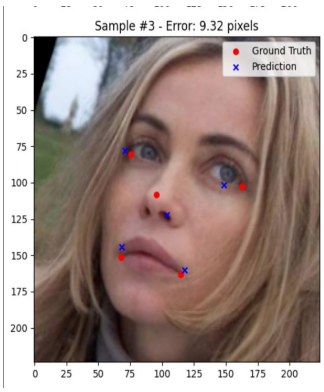
# Summary of Failure Case Studies

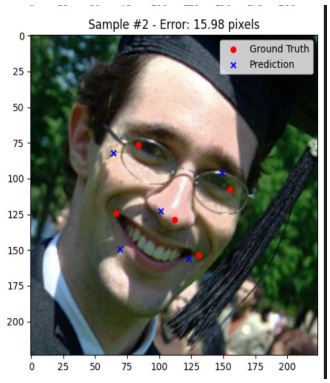
Representative Cases

Case 1 (9.32 pixel error): Slight sideways tilt of the face and uneven illumination lead to biased prediction of the right feature point.

Case 2 (15.98 pixel error): Glasses, hat and smile make it difficult to locate key points.

Critical model deficiencies




Posture sensitivity: Slight changes in facial angle lead to a decrease in prediction accuracy.

Weak adaptation to occlusion: Cannot effectively handle occlusion by accessories such as glasses.

Limitations in expression recognition: poor adaptation to non-neutral expressions

Insufficient structure awareness: 2D CNNs are unable to understand the 3D structure of the face.

Environmental factors: Light variations and complex backgrounds significantly affect prediction results.

Although ResNet performs well under standard conditions, it still has limitations in the face of the complexity of real-world scenarios, and needs to be improved by introducing 3D perception, pose invariance, and occlusion processing.

# Conclusion and Future Outlook

Through this study, we have successfully constructed a ResNet-based face keypoint alignment system with an average Euclidean distance error of about 6.34 pixels and a maximum error reduced to 24.7 pixels. The main innovations of the system are the use of selective backbone network freezing strategy, WingLoss loss function optimisation, and hierarchical learning rate scheduling, which improve the accuracy by about 18% compared to the standard implementation.

Future research will focus on introducing the Transformer attention mechanism to capture global dependencies, adopting more sophisticated data enhancement strategies to cope with diverse real-world scenarios, and exploring model integration and test-time enhancement to improve robustness. This system has the potential for a wide range of applications in the fields of human-computer interaction, AR filtering and affective computing, and after further optimisation, it can effectively support real-time face analysis applications to meet the growing demand for intelligent human-computer interaction.